

```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

rehmang110_petdataset_path = kagglehub.dataset_download('rehmang110/petdataset')
rehmang110_petdatasett_path = kagglehub.dataset_download('rehmang110/petdatasett')

print('Data source import complete.')
```

```
#How to Use SegFormer-B0 from HuggingFace
# install the below package
!pip install transformers datasets torchvision
```

[Show hidden output](#)

```
import torch
import pandas as pd
import numpy as np
from torch.utils.data import Dataset
import torchvision
from pathlib import Path
from PIL import Image
import os
from tqdm import tqdm
import cv2
import shutil
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader
# fetch model from the hugging face
from transformers import SegformerForSemanticSegmentation, SegformerImageProcessor
```

```
print(f"Torch version : {torch.__version__} ")
print(f"pandas version : {pd.__version__}")
```

```
Torch version : 2.6.0+cu124
pandas version : 2.2.3
```

```
if torch.cuda.is_available():
    device = "cuda"
    dtype = torch.float16
else:
    device = "cpu"
    dtype = torch.float16

print(f"Device is :{ device}")
print(f>Data type is :{ dtype}")
```

```
Device is :cuda
Data type is :torch.float16
```

```
#now you have the model and the we have to do now is to prepare a datasets
# Source and destination paths
src_path = Path("/kaggle/input/petdataset/annotations/annotations")
dest_path = Path("/kaggle/working/datasets/annotations")
dest_path.mkdir(parents=True, exist_ok=True)

for item in src_path.iterdir():
    if item.is_file():
```

```
shutil.copy(item, dest_path / item.name)
print(f"Copied file: {item.name}")

elif item.is_dir():
    dest_dir = dest_path / item.name
    dest_dir.mkdir(parents=True, exist_ok=True)

    for file in item.iterdir():
        if file.is_file():
            shutil.copy(file, dest_dir / file.name) # for copying the file
```

[Show hidden output](#)

```
#now you have the model and the we have to do now is to prepare a datasets
# Source and destination paths
src_path = Path("/kaggle/input/petdataset/images/images")
dest_path = Path("/kaggle/working/datasets/images")
dest_path.mkdir(parents=True, exist_ok=True)

for item in src_path.iterdir():
    if item.is_file():
        shutil.copy(item, dest_path / item.name)
        print(f"Copied file: {item.name}")

    elif item.is_dir():
        dest_dir = dest_path / item.name
        dest_dir.mkdir(parents=True, exist_ok=True)

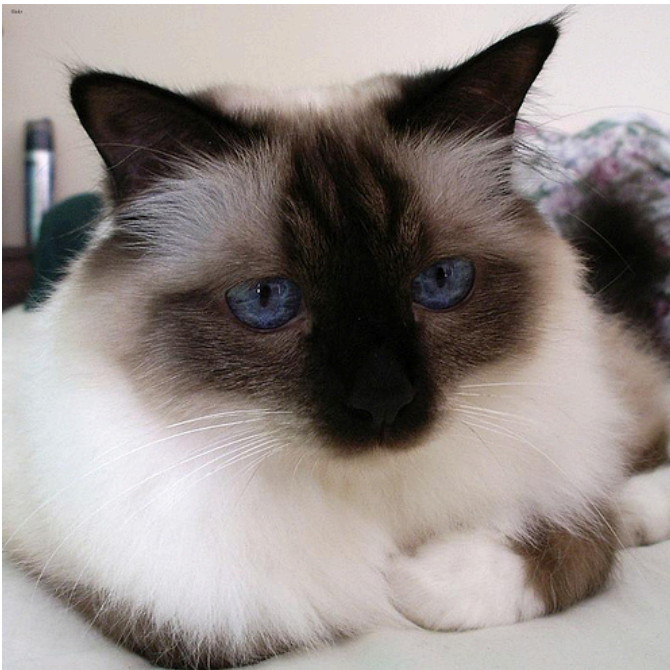
        for file in item.iterdir():
            if file.is_file():
                shutil.copy(file, dest_dir / file.name) # for copying the file
```

[Show hidden output](#)

```
# here we see our dataset how looks like
images_dir = Path("/kaggle/working/datasets/images")
images_list = [item.name for item in images_dir.iterdir()]
print(images_list[:10])
```

```
['samoyed_61.jpg', 'american_pit_bull_terrier_143.jpg', 'shiba_inu_178.jpg', 'havanese_179.jpg', 'Ragdoll_190.jpg',
```

```
image_path = images_dir / "Birman_69.jpg"
Image.open(image_path)
```



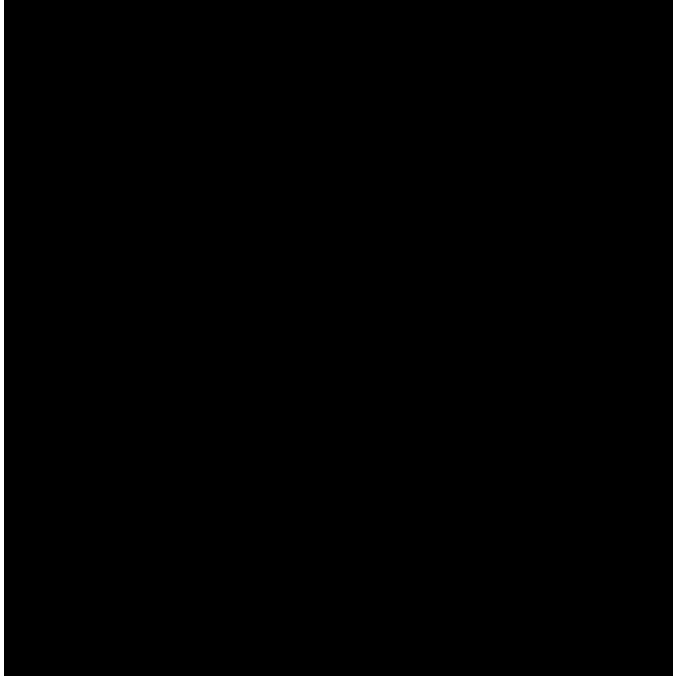
```
# here we see our dataset how looks like
masks_dir = Path("/kaggle/working/datasets/annotations/trimaps")
masks_list = [item.name for item in masks_dir.iterdir()]
print(masks_list[:10])
```

```
['german_shorthaired_55.png', 'great_pyrenees_64.png', 'British_Shorthair_134.png', 'Persian_75.png', '._basset_hou
```

```
mask_path = masks_dir / "Birman_69.png"
```

```
Image.open(mask_path)
```

```
# they all seem black, but it actually look to your eye, because 1,2,3 values are black we do not much differentia
```



```
# now this help us to visualize the mask properly and you clearly see the image and its mask
```

```
# Load mask (as grayscale)
```

```
mask_path = mask_path
```

```
mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)
```

```
# Fix labels: original labels are 1, 2, 3 – we shift them to 0, 1, 2
```

```
mask = mask - 1 # Now: 0 = background, 1 = border, 2 = pet
```

```
# Display with color mapping
```

```
plt.figure(figsize=(5,5))
```

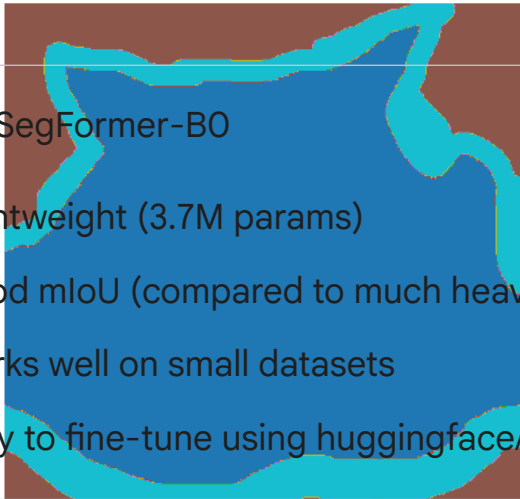
```
plt.imshow(mask, cmap="tab10") # apply a visible color map
```

```
plt.title("Segmentation Mask (colored)")
```

```
plt.axis('off')
```

```
plt.show()
```

Segmentation Mask (colored)



✓ SegFormer-B0

Lightweight (3.7M params)

Good mIoU (compared to much heavier models)

Works well on small datasets

Easy to fine-tune using huggingface/transformers or mmsegmentation

Load model and processor

```
model = SegformerForSemanticSegmentation.from_pretrained("nvidia/segformer-b0-finetuned-ade-512-512")
#It's not a model, just a tool that prepares your data the way the model was trained on.
processor = SegformerImageProcessor.from_pretrained("nvidia/segformer-b0-finetuned-ade-512-512")

model.to(device)
```

[Show hidden output](#)

```
# custom class for datasets and help by base class Dataset from torch
class PetSegmentationDataset(Dataset):
    def __init__(self , img_dir , mask_dir,filenames , processor):
        self.img_dir = img_dir
        self.mask_dir = mask_dir
        self.filenames = filenames
        self.processor = processor

    def __len__(self):
        return len(self.filenames)

    def __getitem__(self , idx):
        # Get the filename (without extension)
        img_name = self.filenames[idx]
        img_path = self.img_dir / f"{img_name}.jpg"
        mask_path = self.mask_dir / f"{img_name}.png"

        # load the image and the mask
        image = Image.open(img_path).convert("RGB")
        mask = Image.open(mask_path)

        # Use processor to preprocess both
        encoded_inputs = self.processor(images=image, segmentation_maps=mask, return_tensors="pt")
        encoded_inputs = {k: v.squeeze(0) for k, v in encoded_inputs.items()} # remove batch dim

        return encoded_inputs
```

```
img_dir = Path("/kaggle/working/datasets/images")
filenames = [p.stem for p in img_dir.glob("*.jpg")] # stem = name without extension
```

```
print(masks_dir)
print(images_dir)
```

```
/kaggle/working/datasets/annotations/trimaps
/kaggle/working/datasets/images
```

```
dataset = PetSegmentationDataset(images_dir ,masks_dir , filenames , processor )
```

What Your Dataset Returns (per sample) { 'pixel_values': Tensor of shape (3, H, W), # The input image 'labels': Tensor of shape (H, W) # The ground truth mask (class indices) }

```
sample = dataset[0]

print(sample.keys())
# → dict_keys(['pixel_values', 'labels'])

print(sample['pixel_values'].shape)
# → torch.Size([3, 512, 512]) # if processor resizes to 512x512

print(sample['labels'].shape)
# → torch.Size([512, 512])

print(torch.unique(sample['labels']))
# → tensor([0, 1, 2, ...]) # the class indices present in that mask
```

```
dict_keys(['pixel_values', 'labels'])
torch.Size([3, 512, 512])
torch.Size([512, 512])
tensor([0, 1, 2])
```

```
img_tensor = sample['pixel_values']
mask_tensor = sample['labels']

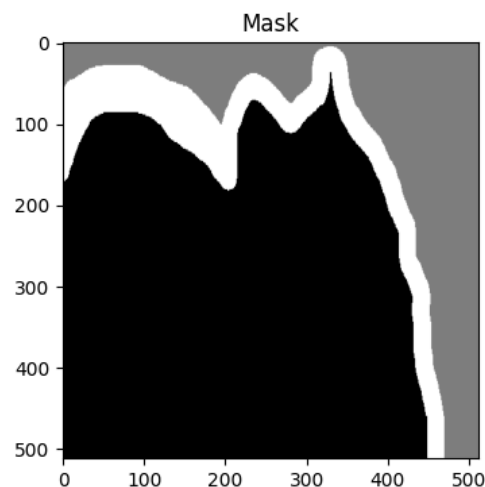
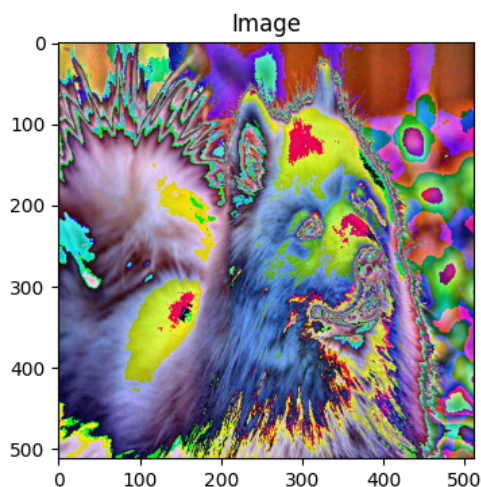
# Convert image tensor to numpy for visualization
img = img_tensor.permute(1, 2, 0).numpy() # (C, H, W) → (H, W, C)
img = (img * 255).astype("uint8") # reverse normalization if needed

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.title("Image")
plt.imshow(img)

plt.subplot(1, 2, 2)
plt.title("Mask")
plt.imshow(mask_tensor, cmap='gray')

plt.show()
```



You are seeing this weird type shape because you the processor of seg former make your pixel values 0-255 , normalize between the 0-1 , to see proper you have to unnormalize again, but we good to go with it.

```
# now create a train and test split ok
train_files, val_files = train_test_split(filenames, test_size=0.2, random_state=42)
print(len(train_files))
print(len(val_files))
```

```
5912
1478
```

```
train_dataset = PetSegmentationDataset(images_dir , masks_dir , train_files , processor )
val_dataset = PetSegmentationDataset(images_dir , masks_dir , val_files , processor )
```

```
train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True, num_workers=2)
val_loader = DataLoader(val_dataset, batch_size=4, shuffle=False, num_workers=2)
```

```
# check the batch
batch = next(iter(train_loader))
print(batch.keys())
print(batch['pixel_values'].shape) # (B, 3, H, W)
print(batch['labels'].shape)      # (B, H, W)
```

```
dict_keys(['pixel_values', 'labels'])
torch.Size([4, 3, 512, 512])
torch.Size([4, 512, 512])
```

SegFormer from HuggingFace already computes the loss when labels are passed in.

```
from torch.optim import AdamW
optimizer = AdamW(model.parameters(), lr=5e-5)
```

```
from tqdm import tqdm

num_epochs = 5

model.train()

for epoch in range(num_epochs):
    total_loss = 0
    loop = tqdm(train_loader, desc=f"Epoch [{epoch+1}/{num_epochs}]")

    for batch in loop:
        inputs = {k: v.to(device) for k, v in batch.items()}

        outputs = model(**inputs)
        loss = outputs.loss
        loss.backward()

        optimizer.step()
        optimizer.zero_grad()

        total_loss += loss.item()

    loop.set_postfix(loss=loss.item())

    avg_loss = total_loss / len(train_loader)
    print(f"✅ Epoch [{epoch+1}/{num_epochs}] Complete - Avg Loss: {avg_loss:.4f}")
```

```
Epoch [1/5]: 100%|██████████| 1478/1478 [07:59<00:00, 3.09it/s, loss=0.143]
✅ Epoch [1/5] Complete - Avg Loss: 0.1562
Epoch [2/5]: 100%|██████████| 1478/1478 [07:57<00:00, 3.10it/s, loss=0.147]
✅ Epoch [2/5] Complete - Avg Loss: 0.1476
Epoch [3/5]: 100%|██████████| 1478/1478 [07:56<00:00, 3.10it/s, loss=0.135]
✅ Epoch [3/5] Complete - Avg Loss: 0.1420
Epoch [4/5]: 100%|██████████| 1478/1478 [07:55<00:00, 3.11it/s, loss=0.203]
✅ Epoch [4/5] Complete - Avg Loss: 0.1355
Epoch [5/5]: 100%|██████████| 1478/1478 [07:54<00:00, 3.11it/s, loss=0.122] ✅ Epoch [5/5] Complete - Avg Loss: 0.
```

```
model.eval()
val_loss = 0

with torch.no_grad():
    for batch in tqdm(val_loader):
        inputs = {k: v.to(device) for k, v in batch.items() }
```

```

        outputs = model(**inputs)
        loss = outputs.loss
        val_loss += loss.item()

print(f"Validation Loss: {val_loss / len(val_loader):.4f}")

100%|██████████| 370/370 [01:03<00:00, 5.84it/s]Validation Loss: 0.1709

```

```

src_images = Path("/kaggle/input/petdatasett")
dest_images = Path("/kaggle/working/test_images")
dest_images.mkdir(parents=True, exist_ok=True)
for file in src_images.iterdir():
    shutil.copy(file, dest_images / file.name)

```

```

folder = Path("/kaggle/working/test_images") # replace with your folder
for i, file in enumerate(sorted(folder.glob("*"))):
    new_name = folder / f"image_{i}.jpg"
    file.rename(new_name)

```

Steps to Evaluate SegFormer & Visualize Predictions

1. Set Model to Eval Mode
2. Load and Preprocess a Single Image
3. Run Inference
4. Convert Prediction to NumPy Mask
5. Visualize Input Image + Predicted Mask
6. Optional: Overlay the Mask on the Image

```

#Set Model to Eval Mode
model.eval()
#Load and Preprocess a Single Image
# Load image
image_path = Path("/kaggle/input/petdatasett/cat3.webp")
image = Image.open(image_path).convert("RGB")

# Preprocess with processor
inputs = processor(images=image, return_tensors="pt").to(device)

```

```

#Run Inferenc
with torch.no_grad():
    outputs = model(**inputs)
    logits = outputs.logits # shape: (1, num_classes, H, W)
    preds = logits.argmax(dim=1) # shape: (1, H, W)

```

```

#Convert Prediction to NumPy Mask
pred_mask = preds.squeeze().cpu().numpy() # shape: (H, W)
#Visualize Input Image + Predicted Mask
import matplotlib.pyplot as plt

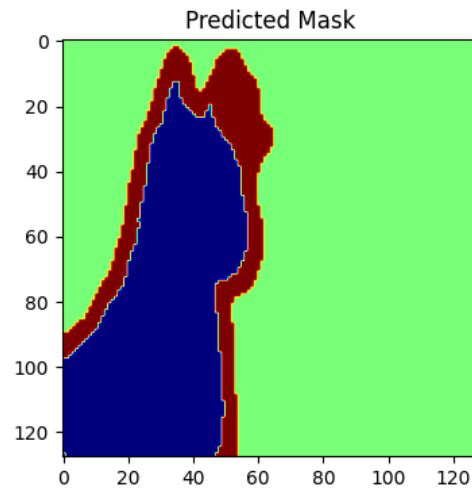
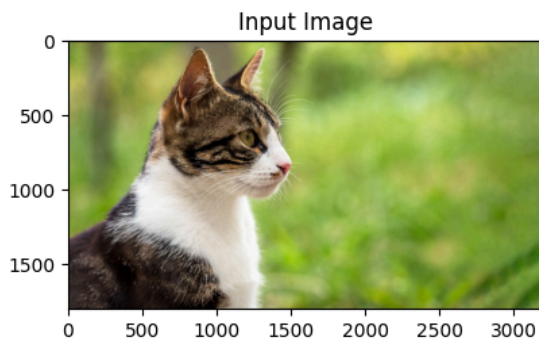
plt.figure(figsize=(10, 4))

# Original Image
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Input Image")

# Predicted Segmentation
plt.subplot(1, 2, 2)
plt.imshow(pred_mask, cmap="jet") # you can try 'viridis' or 'tab20'
plt.title("Predicted Mask")

plt.show()
mask_image = Image.fromarray(pred_mask.astype('uint8'))
mask_image.save("/kaggle/working/predicted_mask_raw.png")
save_path = "/kaggle/working/predicted_mask_aw.png"
plt.imsave(save_path, pred_mask, cmap="jet")

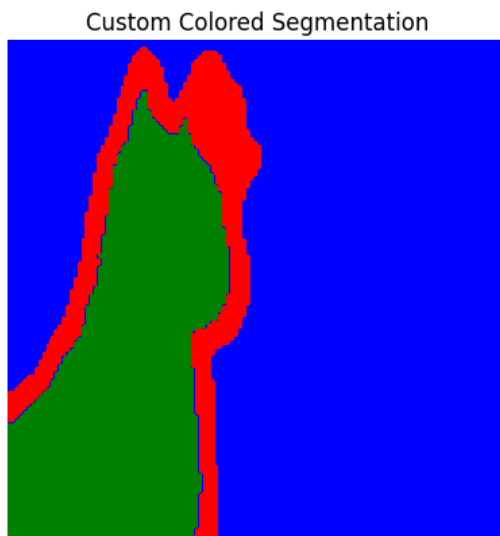
```



```
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

# Define custom color map: [background, cat, border]
colors = ["green", "blue", "red"]
cmap = ListedColormap(colors)

plt.imshow(pred_mask, cmap=cmap)
plt.title("Custom Colored Segmentation")
plt.axis("off")
plt.show()
```



```
import numpy as np
print(np.unique(pred_mask)) # Should show [0, 1, 2]
```

```
[0 1 2]
```

```
#Optional: Overlay the Mask on the Image
import numpy as np

# Resize pred_mask to image size (in case it's smaller)
pred_mask_resized = Image.fromarray(pred_mask.astype(np.uint8)).resize(image.size)

# Convert both to arrays
img_np = np.array(image)
mask_np = np.array(pred_mask_resized)

# Create overlay (semi-transparent)
overlay = img_np.copy()
```



```
overlay[mask_np != 0] = [255, 0, 0] # red for predicted region

blended = (0.6 * img_np + 0.4 * overlay).astype("uint8")

# Show it
plt.imshow(blended)
plt.title("Overlay Prediction")
plt.axis("off")
plt.show()
# Save the current figure
# Convert NumPy array to PIL Image
output_image = Image.fromarray(blended)
# Save to file
output_path = "/kaggle/working/predicted_overlay.png"
output_image.save(output_path)

print(f"✅ Saved overlay to: {output_path}")
```

Overlay Prediction



✅ Saved overlay to: /kaggle/working/predicted_overlay.png

```
model.save_pretrained("segformer-pet-finetuned")
processor.save_pretrained("segformer-pet-finetuned")
```

```
['segformer-pet-finetuned/preprocessor_config.json']
```